PGN : Penalizing Gradient Norm Transfer Attacks in Face Recognition

Chidroopa Kanaparthy

July 4, 2026

Why PGN?

- NeurIPS 2023 publication : Boosting Adversarial Transferability by Achieving Flat Local Maxima
- Official implementation available: github.com/Trustworthy-AI-Group/PGN
- Strong reported gains over the existing baselines
- Not yet implemented in intern baseline

CODE : https://github.com/Chidroopakanaparthy/transferattack/branch/pgn_attack

PGN : Penalizing Gradient Norm

Core idea: adversarial examples sitting in a flat region of the loss surface transfer better across models than ones sitting in a sharp spike, same intuition as flat minima helping generalization in normal training, which here is applied to attacks.

The penalty term needs the Hessian to compute properly, which is too expensive. PGN gets around this with a two-step finite-difference trick instead:

1. Sample a random point x' near the current adversarial image, get gradient g_1
2. Take a small step from x', get gradient g_2 at that new point
3. Combine: $(1-\gamma)\cdot g_1 + \gamma \cdot g_2$ — this approximates the flatness penalty without ever touching the Hessian
4. Repeat N times and average, to kill sampling noise

Then standard momentum on top is the same as MI-FGSM.

Hyperparameters used (repo globals and not paper defaults):
$\varepsilon = 0.062$, $T = 5$, $N = 20$, $\gamma = 0.5$, $\zeta = 3\varepsilon$, $\mu = 1.0$, $\alpha = \varepsilon/T$,

Note on ε:

- paper uses 16/255 ≈ 0.0627 for ImageNet pixel attacks, I used the repo's existing 0.062, so close by coincidence. I Didn't rescale for the face-embedding setting since the repo's other attacks all share this global too,
- Halving T from the paper's default (10→5) to cut runtime doubled this α as a side effect, from the paper's validated ≈6.27e-3 to ours ≈0.0124. Flagged on Slide 9.

Adapting PGN to Face Verification

Problem

Paper's PGN optimizes cross-entropy over classification logits. We need it optimizing cosine similarity between face embeddings instead.

This turned out to be the easy part, the repo already had *attack_loss(cos, attack_type)* handling the impersonation/dodging cosine objective for every other attack. PGN's two-step gradient computation just calls that instead of cross-entropy. No changes were needed to be made to the finite-difference mechanics themselves, g_1 and g_2 come out of two normal *tape.gradient()* calls either way.

Code Integration

Step 1 - Register

```
ALL_ATTACKS = [
  PGD, MI_FGSM, TI_FGSM, SI_NI_FGSM,
  MI_ADMIX_DI_TI, PGN    # <-- added
]
```

Step 2 - Dispatcher

```
if attack_name == "PGN":
    return pgn_attack(...)
```

Reused as-is from the existing baseline attack_loss(cos, attack_type) for the cosine objective, ε-ball projection, and clipping - none of that needed touching.

pgn_attack() - per iteration (T=5)

1. Loop N=20 times:
   - sample x' = adv + random perturbation within ζ-ball (ζ = 3ε)
   - tape.gradient() at x' → g_1
   - step from x' using g_1 → x*, x* = x' − α·(g'/||g'||₁)
   - tape.gradient() at x* → g_2
   - accumulate (1−γ)·g_1 + γ·g_2
2. Average the accumulated gradient over the 20 samples → ḡ
3. Momentum update: g = μ·g + ḡ / ||ḡ||₁
4. adv = adv + α·sign(g), project into ε-ball, clip to valid range

Cost per image: N × 2 gradient evals × T iterations = 200 forward-backward passes.

Cost per subset run: 200 × 30 pairs = 12,000 passes, this is what made the eager-mode runtime a problem

Note: I use γ for the balanced coefficient, the paper calls this δ.

The Real Issues

The math worked on the first try, no NaNs, no shape errors, no logged instability in the two-gradient combination. But the problem was speed.

Running 200 passes/image in normal eager TensorFlow, a full evaluation run passed the 1-hour mark and looked completely dead. There were no error, no crash, just nothing happening. I thought it was a memory leak or a silent failure for a while, but there was zero output with no idea if it was frozen or just slow.

Turned out: ArcFace's graph is heavy enough that eager-mode overhead (the Python ↔ C++ bridge, re-traced every single pass) made it grind to a halt. Facenet512 chugged through the same loop slowly but fine, ArcFace as the attacker model was specifically what broke it.

Fix: wrapped the attack in @tf.function and switched Python range() to tf.range(), so the 200 passes compile into one XLA graph instead of re-tracing every time. Runtime went from ~1.5 hours to ~1 minute. N and T were never reduced. This was purely an execution problem, not a results problem.

Two other bugs along the way:

- `KeyError: 'Facenet'` - eval loop expected Facenet512 as the key, had to fix the mapping
- `CUDA_ERROR_NO_DEVICE` - repo had `CUDA_VISIBLE_DEVICES = -1 ` hardcoded somewhere in the code which silently forced CPU usage, deleting it activated the T4

Final Results

Attack	Breach %	Impact	Dodging %	Impers. %
DPA_HMA	40.21	0.2150	51.25	29.17
PGN (ours)	38.06	0.1885	51.67	24.44
BSR	36.46	0.2048	49.17	23.75
LI_BOOST_MI	35.21	0.2007	46.67	23.75
DeCowA	32.50	0.1931	43.75	21.25
BPA_CNN	30.21	0.1803	40.00	20.42
ATT_CNN	26.67	0.1646	35.00	18.33
SIA_MI_TI	23.33	0.1376	28.75	17.92

2nd of 8. Highest dodging rate on the entire board (51.67%), beats 1-ranked DPA_HMA on that specific axis. Loses overall on impersonation (24.44% vs 29.17%).

Dodging vs Impersonation Split

Dodging consistently outperforms impersonation across almost every attacker-victim pair.

E.g. : VGG-Face → ArcFace: 80% vs 40%, Facenet512 → ArcFace: 80% vs 20%.

One clean exception

ArcFace→Facenet512 flips (20% dodging vs 40% impersonation).

Working theory

Dodging just needs the embedding to cross a similarity threshold in any direction - a flat, spread-out perturbation is well-suited for that. Impersonation needs the embedding to land near one specific target point, a narrower target that flatness doesn't help hit precisely. I Haven't verified this against the actual embedding geometry yet, flagging it as the thing to check before treating it as a real finding rather than a pattern that happens to fit the numbers.

Why Second, Not First

PGN's flatness penalty was built and tuned on classification loss surfaces - ImageNet, cross-entropy, clean class boundaries. We're asking it to do the same trick on cosine similarity between face embeddings, which is a genuinely different loss landscape. The finite-difference step assumes local curvature behaves roughly like cross-entropy does; nobody's actually checked whether that holds for embedding space, and I haven't verified it here either, it's just the most likely explanation on the table.

What makes me somewhat confident the core mechanism is still doing real work: PGN beat SIA_MI_TI, ATT_CNN, and BPA_CNN comfortably, and posted the single best dodging number on the entire leaderboard (51.67%, ahead of DPA_HMA's own 51.25%). It's specifically losing ground on impersonation (24.44% vs DPA_HMA's 29.17%).

Two honest possibilities, not fully disentangled yet:

- The flatness assumption genuinely transfers worse to a ‘hit a specific target’ objective (impersonation) than a ‘leave a region’ objective (dodging), ties back to the slide 8 pattern
- Or it's just a hyperparameter fit issue - γ=0.5 and ζ=3ε were carried over unchanged from the paper's ImageNet setup, never retuned for this task, so there's real room DPA_HMA's method might just be better-calibrated to this specific pipeline rather than mechanistically superior
- Or the finite-difference step size drifted out of the paper's tested range. α=ε/T is reused inside the Hessian approximation itself (Algorithm 1, step 7). The paper's ablation (Appendix D.2) tunes this specifically and finds smaller values degrade transferability by collapsing the method toward plain MI-FGSM - their validated value is α≈6.27e-3. Reducing T from 10→5 to cut runtime doubled α to ≈0.0124 as a side effect. The paper didn't test larger step sizes, but Theorem 1's derivation assumes α is small for the Taylor expansion to hold, so a larger α plausibly means more approximation error in the flatness penalty itself, independent of the classification, cosine-similarity gap above.

I'd want to rerun with a δ/ζ sweep and a T=10 pass to isolate the α effect before pinning the gap on any one of these three.

What I Learned

Technical

- The actual trick in PGN - approximating a Hessian-based penalty with two nearby gradient evaluations instead of computing it directly, it is a nice general pattern, not something I'd seen used this way before
- Eager-mode TensorFlow overhead compounds badly with nested sampling loops - 200 passes/image doesn't sound like much until each one re-traces the Python ↔ C++ bridge individually
- @tf.function made the running process go from 1.5 hours to ~1 minute changed the whole feasibility of even getting to results

Process

- Learned to actually distinguish between what is broken to what is just slow before reacting, the 108 minutes of silent output genuinely had me ready to assume something was structurally wrong and go debug the wrong layer
- ArcFace specifically being the one that broke eager mode was a good reminder that the code ran fine on one model doesn't mean it'll run fine on all of them, model complexity itself is a variable worth testing across
- Extending someone else's baseline without touching its structure is a completely different skill from writing something from scratch.

Thank you

Chidroopa Kanaparthy